

Cloud Config and Discovery: Consul에서 Nacos 기반 DUCOS로

설정 저장소가 아니라, tenant·stage·override·runtime refresh를 통제하는 운영 구성 계층

요약

초기에는 Consul 기반으로 service discovery와 config를 운영했다. 서비스가 늘고 고객사별 설정, stage별 차이, runtime refresh, 운영 승인, 장애 추적 요구가 커지면서 단순 key-value 저장 구조만으로는 부족해졌다. 이후 Nacos를 기반으로 DUCOS라는 운영 구성 계층을 설계했다.

DUCOS의 목적은 Nacos를 감싸는 wrapper가 아니었다. 설정의 의미, 적용 범위, 변경 승인, refresh 위험, discovery 상태, 장애 추적을 하나의 운영 모델로 다루는 것이 핵심이었다.

문제 정의

운영 설정은 단순한 값이 아니다. ERP·Groupware·ESS 환경에서는 고객사, 업무 모듈, stage, 배포 버전, 외부 연동 상태에 따라 설정의 의미가 달라진다. 같은 key라도 dev와 prod에서 의미가 다르고, tenant override가 적용되면 기본값과 실제값이 달라진다.

초기 Consul 구조는 discovery와 간단한 설정 관리에는 유효했다. 그러나 다음 요구가 쌓이면서 별도 운영 계층이 필요해졌다.

- tenant별 override와 기본값의 resolution 순서를 설명해야 한다.
- 설정 변경이 어떤 서비스와 고객사에 영향을 주는지 알아야 한다.
- runtime refresh를 허용할 설정과 재배포가 필요한 설정을 구분해야 한다.
- 잘못된 설정 변경을 감사하고 rollback할 수 있어야 한다.
- discovery는 주소 목록이 아니라 서비스 상태와 운영 가능성을 함께 표현해야 한다.

책임 경계

계층	책임
Nacos	config 저장, naming, service discovery 기본 기능을 제공한다.
DUCOS	tenant, stage, namespace, override, approval, audit, refresh 정책을 정의한다.

계층	책임
Application	설정을 읽고 적용하되 resolution 규칙을 임의로 재해석하지 않는다.
운영 UI	실제 적용값, 상속 경로, 변경 이력, 영향 범위를 보여준다.
장애 모니터링	설정 변경과 장애 이벤트를 연결한다.

핵심 설계 결정

namespace보다 이름의 의미를 먼저 정했다

namespace를 과도하게 늘리면 분류는 세밀해지지만 운영자가 실제 적용값을 이해하기 어렵다. DUCOS에서는 namespace, group, dataId를 기술적 분류가 아니라 운영 의미와 연결했다. 고객사, stage, 업무 도메인, 설정 종류가 이름에 드러나야 했다.

override는 편의 기능이 아니라 위험한 변경으로 다뤘다

tenant override는 영향 범위가 큰 도구이며 운영 위험도 크다. 기본값과 override가 섞이면 실제 적용값을 사람이 머릿속으로 계산해야 하는 상황이 생긴다. 그래서 DUCOS에서는 resolution 결과를 화면에 명확히 보여주고, 어떤 레벨의 설정이 최종값이 되었는지 기록하도록 설계했다.

```
default
-> stage default
  -> tenant default
    -> tenant-stage override
      -> emergency override
```

이 순서가 코드마다 다르게 구현되면 운영 통제는 깨진다. resolution 규칙은 플랫폼의 계약으로 다뤄야 한다.

runtime refresh는 허용 목록 기반으로 제한했다

모든 설정을 runtime refresh 대상으로 두면 배포 없이 빠르게 바꿀 수 있다. 그러나 DB pool, security, transaction, serialization, scheduler 같은 설정은 실행 중 변경 시 장애를 만들 수 있다. 따라서 refresh 가능한 설정과 재시작이 필요한 설정을 분리했다.

runtime refresh는 기능이 아니라 운영 위험을 동반한 변경이다. 변경 전 검증, 적용 대상 확인, 실패 시 rollback, 적용 이력 기록이 함께 있어야 한다.

버린 선택과 이유

Consul이나 Nacos의 기본 기능만 그대로 사용하는 선택도 가능했다. 그러나 config 저장소가 값만 보관하면 운영자는 실제 적용값을 다시 계산해야 한다. tenant override, stage default, 긴급 차단 설정이 겹치면 “최종 값이 무엇인가”가 운영 리스크가 된다.

application 내부에서 설정 resolution을 처리하는 방식도 제외했다. 각 서비스가 자체 규칙을 가지면 동일한 설정이 서비스마다 다르게 해석될 수 있다. 운영 설정은 application 코드의 편의 기능이 아니라 중앙에서 설명 가능한 계약이어야 했다.

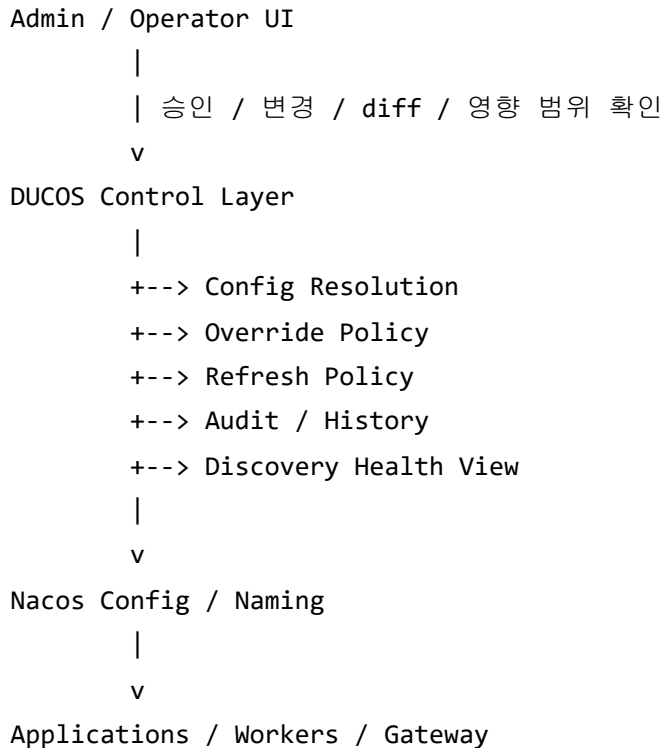
runtime refresh를 자유롭게 허용하는 방식도 피했다. refresh는 재배포 없이 값을 바꿀 수 있어 편리하지만, DB pool, thread, cache, 외부 연동 endpoint처럼 runtime 변경이 위험한 항목도 있다. 그래서 refresh 가능 설정과 재기동이 필요한 설정을 명시적으로 구분했다.

설정 변경을 장애와 연결한 이유

설정 변경 이력이 장애 이벤트와 분리되면 운영자는 장애 시점에 어떤 값이 바뀌었는지 따로 추적해야 한다. DUCOS에서는 설정 변경을 단순 audit으로만 남기지 않고, 고객사·stage·서비스·배포 버전·장애 이벤트와 연결 가능한 형태로 남기는 것을 목표로 했다.

특히 다중 고객사 환경에서는 기본 설정이 정상이어도 특정 고객사의 override만 장애를 만들 수 있다. 따라서 “전체 서비스 정상”이라는 판단보다 “어느 고객사에 어떤 override가 적용되었는가”를 확인하는 기능이 중요했다.

아키텍처



DUCOS는 Nacos의 기능을 숨기지 않는다. 대신 운영자가 설정을 안전하게 바꿀 수 있도록 resolution, approval, audit, refresh 정책을 추가한다. application은 Nacos 값을 직접 해석하지 않고 DUCOS의 계약에 맞춰 설정을 소비한다.

discovery를 바라본 기준

Discovery는 단순히 주소 목록을 반환하는 기능이 아니다. 운영에서는 service instance가 살아 있는지, 어떤 stage에 속하는지, 어떤 version인지, 어떤 tenant traffic을 받을 수 있는지, health 상태가 어떤지 확인해야 한다.

따라서 discovery 정보는 routing, canary, 장애 대응, 배포 전환과 연결되어야 한다. OpenResty L7, 배포 시스템, 장애 모니터링과 분리된 주소 목록으로 두면 운영 판단이 늦어진다.

운영 검증

가장 중요한 검증은 “실제 적용값을 설명할 수 있는가”였다. 운영자는 key 이름만 보고 판단하지 않는다. 기본 값, stage 값, tenant override, emergency override가 어떤 순서로 합쳐졌는지 확인해야 한다.

두 번째 검증은 refresh 실패 경로였다. refresh 가능한 설정도 적용 실패, 일부 인스턴스 미반영, 이전 값과 신규 값의 불일치가 발생할 수 있다. DUCOS는 적용 상태와 실패 대상을 추적하고, 장애 모니터링 이벤트와 연결되도록 설계했다.

세 번째 검증은 설정 변경과 장애의 상관관계였다. 특정 설정 변경 이후 특정 고객사 장애가 증가하면 두 정보를 같은 화면에서 확인할 수 있어야 한다.

역할과 책임

Consul 기반 초기 구조의 한계를 분석하고, Nacos 기반 DUCOS 운영 모델, namespace/group/dataId 규칙, tenant override resolution, runtime refresh 정책, audit 구조, discovery 운영 화면의 방향을 설계했다.

설계 결론

Config와 discovery는 인프라 기능이지만 운영 책임은 application에 직접 영향을 준다. 설정은 값이 아니라 변경 가능한 운영 정책이다. DUCOS의 핵심은 Nacos 도입이 아니라, 설정 변경을 설명 가능하고 되돌릴 수 있는 운영 행위로 만든 것이다.